



Escuela Superior de Computo "ESCOM"

Documento técnico

Landeros Cortes Pedro Jonas

Desarrollo de aplicaciones móviles nativas

Examen a Título de Suficiencia (ETS) — Proyecto
práctico

Profesor: GABRIEL HURTADO AVILÉS

Documento técnico

1. Objetivo y alcance

Objetivo. Implementar una aplicación Android nativa en Kotlin que permita a un dispositivo servidor obtener o preparar videos y enviarlos por Bluetooth a un dispositivo cliente sin acceso directo a Internet. El documento resume decisiones de diseño, arquitectura y protocolo de comunicación. La implementación prioriza compatibilidad, trazabilidad del flujo y una demostración clara en dos teléfonos físicos.

2. Arquitectura cliente-servidor

Elemento	Detalle
Cliente Android	Activity de cliente con búsqueda, selección de fuente, recepción de metadatos y chunks, almacenamiento temporal, VideoView, historial y favoritos SQLite.
Servidor Android	Activity de servidor con socket RFCOMM, biblioteca seleccionada desde el dispositivo, catálogo demo y fuente YouTube experimental.
Capa Bluetooth	BluetoothConnection encapsula listenUsingRfcommWithServiceRecord, createRfcommSocketToServiceRecord, lectura/escritura de frames y cierre de socket.
Capa de protocolo	Protocol define tipos de frame, payloads JSON, tamaño de chunk y UUID de servicio.
Fuentes de video	ServerVideoLibrary/VideoCatalog para videos locales; YoutubeExperimentalSource para búsqueda/preparación experimental con yt-dlp/FFmpeg.
Persistencia	LocalStore usa SQLiteOpenHelper para historial y favoritos en el cliente.

Flujo de alto nivel: el servidor queda en escucha, el cliente se conecta al UUID de servicio, el cliente envía búsqueda, el servidor responde resultados, el cliente solicita reproducción, el servidor envía metadata y chunks, el cliente reconstruye el archivo y reproduce localmente.

3. Protocolo Bluetooth

Se usa Bluetooth Classic RFCOMM para simular un canal serial confiable entre dos dispositivos emparejados. La app no usa BLE porque BLE no es adecuado para transportar archivos de video grandes. El servicio se identifica con el nombre BtVideoPlayer y el UUID 7f67a2ac-4d64-4cb1-81f0-2cda9be39981.

Elemento	Detalle
Estructura de frame	Cada frame escribe primero typeCode como Int de 4 bytes, después length como Int de 4 bytes y finalmente payload de longitud length.
Límite de seguridad	El receptor rechaza frames con longitud negativa o mayor a 25 MB para evitar lecturas inválidas.
Chunk de video	El video se divide en bloques de 16 KB. Esto reduce memoria y permite reportar progreso de buffer.
Payload JSON	Las solicitudes de búsqueda/reproducción, metadatos y errores viajan como JSON UTF-8.
Payload binario	VIDEO_CHUNK transporta bytes crudos del archivo preparado por el servidor.

4. Tipos de frame y responsabilidad

Frame	Dirección	Uso
SEARCH_REQUEST	Cliente -> Servidor	query + sourceMode.
SEARCH_RESPONSE	Servidor -> Cliente	Lista de SearchResult serializada en JSON.
PLAY_REQUEST	Cliente -> Servidor	videoid + sourceMode + lowPower.

Frame	Dirección	Uso
VIDEO_META	Servidor -> Cliente	videoid, título, bytes totales y MIME.
VIDEO_CHUNK	Servidor -> Cliente	Fragmento binario del archivo.
VIDEO_END	Servidor -> Cliente	Fin de transferencia; el cliente cierra archivo y reproduce.
ERROR	Ambas	Mensaje JSON con descripción de fallo.
PING/PONG	Reservado	Tipos definidos para extensiones de salud/conectividad.

5. Decisiones de implementación

- Kotlin nativo y XML: reduce dependencias y mantiene compatibilidad amplia con Android 8.0+.
- VideoView: solución simple para reproducir archivos locales recibidos; requiere codecs compatibles del dispositivo.
- TransferVideo abstrae nombre, tamaño, MIME y stream, permitiendo enviar videos desde assets, documentos del servidor o caché YouTube.
- Servidor con Document Picker: evita permisos amplios de almacenamiento y guarda permisos persistentes por URI.
- YouTube experimental: el servidor prepara MP4/H.264/AAC con yt-dlp/FFmpeg, limita resolución a 360p en bajo consumo y 480p normal.
- Historial/favoritos en SQLite: persistencia local simple, offline y testeable sin Bluetooth.
- Modo privado: evita escribir historial para cumplir un caso de privacidad básico.

6. Compatibilidad, pruebas y limitaciones

Elemento	Detalle
Compatibilidad Android	minSdk 26, targetSdk 35, permisos separados para Android 11- y Android 12+.
Hardware mínimo	Servidor: Bluetooth 4.1+, Internet estable preferentemente Wi-Fi y espacio para caché. Cliente: Bluetooth compatible, sin Wi-Fi ni datos durante la prueba.
Pruebas	Tests instrumentados para YouTube y LocalStore; validación Bluetooth manual con dos dispositivos físicos.
Límites de reproducción	MP4/H.264/AAC recomendado; YouTube limitado a 360p/480p; no se soportan DRM, privados, lives ni videos restringidos.
Rendimiento	Bluetooth es lento frente a Wi-Fi; la app transfiere chunks y reproduce al finalizar, no implementa streaming adaptativo.
Riesgos externos	YouTube puede cambiar formatos/extractores; el modo experimental puede requerir actualización de yt-dlp o fallar con videos específicos.